公众号文章 → Word 自动化工具

从零搭建 · 启动运行 · 请求处理 · 质量保障 全流程拆解

✅ 本机制服已跑通：http://localhost:3000（HTTP 200）

1 用户需求与解决痛点

先看清"谁在用、为什么用"，后面每一步搭建才有落点。

👤 内容 / 出版编辑

要把已授权的公众号文章整理成符合出版规范的 Word 稿件

👤 选题 / 汇编负责人

手里只有标题清单，需要逐篇定位原文并合成一本汇编

👤 团队同事

零安装，浏览器打开就能用（同一局域网共享一套服务）

👤 工具搭建者

只需部署一次，其余人直接用

📌 需求一：单篇归档

- 拿到一条微信链接 → 出一篇排版规范的 Word
- 保留原图文顺序，图片按原文位置嵌入
- 文件名统一为 `文章标题（类别）.docx`

📌 需求二：批量归档

- 一次粘贴几十条链接 → 批量出稿并打包
- 必须附带 `可追溯清单`：原名、新文件名、作者、来源、链接、失败原因
- 个别失败不中断整体

📌 需求三：选题汇编

- 只有 `公众号：xxx + 标题列表`（选题会片单），没有链接
- 自动搜索定位每篇链接 → 合成 `一个` Word
- 未成功的篇目能一键续跑，不重复劳动

❗ 手工做这件事的 10 个痛点，以及工具给出的对应解法：

#	😠 人工操作的痛点	✅ 工具的解法
1	**复制粘贴丢排版** 图文错位、样式全丢，正文要重排	**结构化解析** `parser.js` 递归遍历正文，还原标题/段落/图片/图注/参考文献
2	**图片全靠手动** 逐张另存、逐张插入、逐张调宽度	**自动下载并嵌入** 并发 6 路下载，统一图宽 14.7cm，同文重复图只下一次
3	**噪声删不干净** 顶部二维码、底部推广、留言区混进正文	**自动降噪并标注** `qrDetector.js` 识别二维码，噪声剔除处以红色标注，便于人工复核
4	**排版格式不统一** 每篇字号行距都靠手调，篇篇不一样	**规则固化** 主标题 24pt / 章节 20pt / 正文 12pt 宋体 / 参考文献 9pt 灰
5	**只有标题没有链接** 人工一篇篇搜，最耗时的一环	**搜索定位** 本地链接库优先 → 联网搜索回落 → 结果缓存复用（`.cache`）
6	**批量无台账** 哪篇失败、为什么失败全靠记忆	**清单自动生成** zip 内附 Excel + 纯文本清单，含失败原因
7	**失败要从头再来** 一篇卡住就得整批重做	**一键续跑** 只重跑失败篇目，已定位的走本地库不重复搜索
8	**版权图源漏标** 视觉中国等图源没提示，出版有风险	**红色版权提醒** 命中"视觉中国 / 东方IC / 人民视觉 / IC photo"即标注
9	**视频音频不好处理** Word 里嵌不了，各人做法不一致	**统一转链接备注** 不嵌入、不丢信息
10	**命名口径不一** 汇总核对时找不到对应关系	**文件名规范化** 非法字符清洗 + `文章标题（类别）.docx`，公众号名统一映射

2 搭建部署主流程

从拿到压缩包到浏览器可访问，共 6 步；第 4 步有两条启动路径，任选其一。

1 准备运行环境

安装 **Node.js ≥ 18**（推荐 20），自带 npm。首次安装依赖需联网。

校验：`node -v` · `npm -v`

2 获取项目代码

解压 `wechat-to-word_20261010.zip` → 得到 `wechat-to-word_handover_20260911/`

入口文件：`server.js` · `package.json` · `public/index.html` · `src/` · `config/`

3

安装依赖（首次必做）

`npm install` → 生成 `node_modules/`（约 200 个包，1–2 分钟）

核心依赖：`express` Web 服务 · `cheerio` HTML 解析 · `docx` Word 生成 · `adm-zip` 打包 · `jimp` 图像处理 · `jsqr` 二维码识别

第 4 步 · 两种启动方式，任选其一

方式 A：双击启动（macOS 推荐）

① 双击 `启动工具.command`

若提示"无法打开"，先执行一次 `chmod +x 启动工具.command`

▼

② 脚本自动探测 Node.js

依次查找 PATH → /usr/local/bin → /opt/homebrew/bin → nvm → .workbuddy

▼

③ 无 node_modules 则自动 `npm install`

也有 npm 则一并校验

▼

④ 取局域网 IP 并打印访问地址

`ipconfig getifaddr en0`

▼

⑤ 延时 2s 自动打开浏览器

随后前台运行 `PORT=3000 node server.js`

方式 B：命令行启动（全平台通用）

① 进入项目目录

`cd wechat-to-word_handover_20260911`

▼

② 安装依赖

`npm install`（仅首次）

▼

③ 启动服务

`npm start` 等价 `node server.js`

▼

④ 可选：并发调优

`IMAGE_CONCURRENCY=8 ARTICLE_CONCURRENCY=4 npm start`

默认 6 / 3，已按"不被微信限流"取值

▼

⑤ 手动打开浏览器

访问 `http://localhost:3000`

5 访问网站

本机：`http://localhost:3000`

同事共用（同一局域网）：保持窗口不关，对方访问 `http://你的局域网IP:3000`（服务监听 `0.0.0.0`）

6 验证可用

页面出现并列三栏：🧠 **轻心理** · 📰 **轻纪实** · 🗂️ **汇编**（汇编内含"链接汇编 / 🔍 搜索汇编"两个子页签）→ 搭建完成

注意：服务为前台进程，关闭终端窗口即停止；`.gitignore` 已忽略 `node_modules/`、`.cache/`、`输出预览/`，交付时无需打包这些目录。

3 运行时架构（分层）

单进程 Express 应用：静态页面 + 4 个 API，无数据库，状态落在 `.cache/` 与 `config/` 的 JSON 文件里。

接入层 · 浏览器

public/index.html
三栏工作区：轻心理 / 轻纪实 / 汇编（液态玻璃 UI，背景 `assets/bg.png`）

public/map.html
附带的地图页（ECharts + `china.json` / `cities/`）

服务层 · SERVER.JS（EXPRESS 5）

express.static(public)
托管前端页面与静态资源

express.json()
解析 POST JSON 请求体

PORT / HOST
3000 / 0.0.0.0（局域网可达）

接口层 · 4 个 REST API

POST /api/convert
单篇：一个链接 → 一个 Word

POST /api/convert-batch
批量：多个链接 → zip（含 Excel 清单）

POST /api/assemble
汇编：多个链接 → 合成一个 Word

POST /api/assemble-search
搜索汇编：公众号+标题 → 定位链接并汇编

业务模块层 · SRC/

fetcher.js
抓取文章 HTML、下载图片（20s 超时）

parser.js
递归遍历正文 → 结构化块（标题/段落/图片/图注/参考文献/噪声）

qrDetector.js
jsQR 识别二维码，保守判定不误删

imageSize.js
只读文件头取宽高，不加载图形库

articleRules.js
版权关键词（视觉中国/东方IC/人民视觉）与图注判定

textStyle.js
字号/加粗/颜色等样式抽取与标题归并

docxBuilder.js
按排版规则生成 .docx（单篇 / 汇编两种）

xlsx.js
生成抓取结果 / 搜索定位结果 Excel 清单

concurrency.js
固定并发度执行、结果保序

wechatSearch.js
标题归一化匹配、请求间隔控制

searchSources.js
可插拔搜索源（内置搜狗微信）

accountNames.js
公众号名规范化映射

↓

数据 / 缓存层

.cache/search-links.json
搜索定位结果缓存（"公众号|标题" → 链接）

config/links.json
本地已知链接库

config/account-names.json
公众号名映射表

config/search-sources.example.json
搜索源配置样例

↓

输出层 · 浏览器下载

.docx
`文章标题（类别）.docx`

.zip
批量/汇编产物

.xlsx + 清单.txt
原名/新文件名/作者/来源/链接/失败原因

待重跑清单.txt
未成功篇目，供一键续跑

4 单篇 / 批量抓取处理流水线

对应 `/api/convert` 与 `/api/convert-batch`，是一条"链接进、Word 出"的直线流水线。

输入

粘贴链接

单篇一条，批量多条

server.js

修复断行链接

`repairBrokenUrls`

拼接被换行截断的长参数链

server.js

提取并去重

`extractWechatLinks`

兼容 `/s/xxx` 与 `/s?__biz=`

concurrency

并发抓取文章

ARTICLE_CONCURRENCY = 3

fetcher

抓取 HTML

带超时与重试

parser

解析成结构化块

标题/正文/图片/图注/参考文献/噪声

fetcher+imageSize

并发下载图片

IMAGE_CONCURRENCY = 6，同文重复图只下一次

qrDetector

降噪

顶部二维码、底部推广、留言区剔除并红色标注；视频音频转链接备注

articleRules

版权检查

命中视觉中国/东方IC/人民视觉 → 红色提醒

docxBuilder

排版生成 Word

主标题 24pt / 章节 20pt / 正文 12pt 宋体；图宽 14.7cm；参考文献 9pt 灰

adm-zip + xlsx

打包下载

单篇直接下载 docx；批量打包 zip + Excel 清单

5 搜索汇编流水线（只有标题、没有链接时）

对应 `/api/assemble-search`，比第 3 条多出"搜索定位"与"一键续跑"两个环节。

输入

公众号 + 标题列表

"公众号：人物"
下面逐行写标题（可带类别）

parser

拆条目

按行解析出（公众号, 标题, 类别）

searchSources

查本地链接库

`config/links.json` + `.cache` 命中即直接用

searchSources

未命中则联网搜索

搜狗微信等可插拔数据源

wechatSearch + accountNames

匹配确认

标题归一化比对、公众号名规范化

→ 第 4 节

抓取 + 排版

复用同一条流水线

docxBuilder

合成一个 Word

多篇文章按序整合

adm-zip + xlsx

打包

汇编 Word + 搜索定位结果 Excel

前端

未成功篇目回显

页面可直接复制，zip 内含 `待重跑清单.txt`

一键续跑

只重跑失败项

已定位的走本地链接库，不重复搜索

6 网站实操连续性（一次出版任务的完整闭环）

关键不在"能跑一次"，而在每一步的产物都能直接喂给下一步，中途不换工具、不重新整理数据。

0 选题会确定片单 策划 / 主编

手里只有"哪个公众号、哪些标题"，通常还没有链接

产物：公众号名 + 标题清单（可带类别）

↓

1 打开网站，进入「汇编」栏 → 搜索汇编 编辑

三栏同屏、不跳转：轻心理 / 轻纪实 / 汇编并列，粘贴清单后点「搜索定位并汇编成一个 Word」

产物：一条请求 POST /api/assemble-search

↓

2 自动定位 + 抓取 + 排版（页面实时进度）

本地链接库优先 → 未命中走搜索源 → 命中后复用第 4 节流水线，多篇合成一个 Word

产物：zip = 汇编 Word + 搜索定位结果 Excel

↓

3 页面直接回显「未成功篇目」→ 点 一键续跑

清单可一键复制粘贴回输入框；已定位的走本地链接库，**不重复搜索**，只补跑失败项

产物：补齐后的 Word + zip 内「待重跑清单.txt」

↓

4 仍定位不到的，人工补链接并沉淀 编辑

把找到的链接补进 `config/links.json`，成为本地链接库的一部分

产物：本地库 +1，下次零成本、零限流

↓

5 编辑在 Word 上做内容加工 编辑

复核红色标注处（被剔除的二维码/推广/留言）、确认图片与版权提醒、改写润色

产物：初排版稿件（已省掉全部机械劳动）

↓

6 进入既有出版流程：三审三校 → 定稿 → 归档

工具止步于"素材获取 + 初排版"，判断与把关仍由人完成

产物：可出版稿件 + 全程留痕的清单台账

🔗 让这条链不断掉的四处设计：

页面不跳转
三栏并列同屏，批量/汇编/续跑都在一个界面内完成

产物即输入
zip 内清单 → 页面上「未成功篇目」→ 直接粘回输入框续跑

缓存可复用
`.cache/search-links.json` 记住已定位结果，重跑不再联网搜索

服务常驻共享
同一局域网多人接力同一任务，不用各自装环境

7 优化出版工作流（Before → After）

工具替换掉的是"机械劳动"，不是"编辑判断"——这决定了它在流程里的位置。

😖 引入工具前：人工流水线

- 打开链接 → 全选复制 → 粘进 Word，样式全丢要重排
- 图片逐张右键另存 → 逐张插入 → 逐张拖宽度
- 手动找出并删除顶部二维码、底部推广、留言区
- 字号、行距、标题层级每篇手调，篇篇不一致
- 只有标题时，人工一篇篇搜索定位原文
- 批量做完没有台账，失败原因全靠记忆
- 一篇失败往往整批重来
- 文件名随手起，汇总核对靠肉眼
- 每人各装一套环境，版本容易不一致
- 找过的链接用完即弃，下次重新找

✅ 引入工具后：自动化流水线

- 粘贴链接即出稿，结构与原文一致
- 图片并发下载并嵌入，统一图宽 **14.7cm**，重复图只下一次
- 自动降噪，剔除处以**红色标注**供人工复核
- 排版规则固化：24 / 20 / 12pt、参考文献 9pt 灰
- 本地链接库优先 + 搜索源回落，自动定位
- zip 内附 Excel 清单：原名、新名、作者、来源、链接、**失败原因**
- 一键续跑，只补跑失败项
- 文件名自动规范化，公众号名统一映射
- 一人部署，局域网浏览器即用，零安装
- 链接沉淀进 **config/links.json** 与缓存，越用越省

约 1–2 min	×3 并发	×6 并发
单篇耗时（原 20–30 min）	批量同时处理文章数	单篇内图片同时下载数

只跑失败项	1 → N
续跑范围（原：整批重来）	一次部署，全组可用

🧭 工具在整个出版流程中的落位与边界：

流程阶段	承担者	说明
① 选题策划	人工	确定选题、公众号与片单——机器不参与
② 素材获取	🛠️ **工具**	定位链接、抓取正文与图片、降噪、初排版（本工具核心价值）
③ 内容编辑	人工	复核红色标注、确认图源版权、改写润色
④ 审校把关	人工	三审三校、事实核查——不可自动化
⑤ 出版归档	人工 + 工具	出版归档；清单台账仍可用于回溯核对

边界说明：工具只做"素材获取 + 初排版"这一段机械劳动，并主动把**不确定的地方标红**（噪声剔除处、版权图源、定位失败项）交回给人判断。它不替代三审三校，也不自动发布——这样既提效，又不把出版风险交给机器。

8 质量保障与持续集成（可选但已内置）

本地自测 + GitHub Actions 回归，改动后先跑测试再交付。

A 本地测试

`npm test` 跑 10 个测试文件（parser / docx / fetcher / imageSize / concurrency / search / searchSources / xlsx / pipeline / risk）

另有 `npm run test:risks`（风险项）、`npm run test:online`（联网用例）、`npm run test:all`

B CI 自动回归

`.github/workflows/regression.yml`：push / PR / 手动触发 → ubuntu-latest + Node 20 → `npm ci` → `npm run test:report` → 上传 `test-results.xml`（失败也上传）

C 风险防控设计

二维码识别**保守判定**（解码失败一律不删）· 并发度**限流**避免被微信封 · 抓取失败写入清单而非中断整体流程

9 命令速查表

照着从上往下执行即可完成搭建。

顺序	命令	说明
1	`node -v`	确认 Node ≥ 18（推荐 20），没有就去 nodejs.org 装 LTS
2	`unzip wechat-to-word_20261010.zip`	解压得到项目目录
3	`cd wechat-to-word_handover_20260911`	进入项目根目录（所有命令都在此执行）
4	`npm install`	安装依赖，仅首次需要；或双击 `启动工具.command` 自动完成
5	`npm start`	启动服务（等价 `node server.js`），终端保持不关
6	打开 `http://localhost:3000`	看到三栏界面即成功
7	`ipconfig getifaddr en0`	macOS 查局域网 IP，供同事访问 `http://IP:3000`
8	`npm test`	可选：改动代码后跑回归测试

基于项目实际源码（server.js / public/index.html / src/*.js / package.json / 使用说明.md / .github/workflows）整理绘制